

Nexus Loop

An Autonomous Detect-Diagnose-Prescribe-Verify Loop for AI-Agent Deployments

Team Name: Team Shekhar

Institution: IIT Patna

Abstract

We present **Nexus Loop**, an autonomous improvement loop for AI-agent deployments. It turns eight weeks of execution logs into diagnosed regressions and verified fix proposals, following a **Detect** → **Diagnose** → **Prescribe** → **Verify** workflow: it mines each deployment’s own definition of “good,” flags cohorts that deviate from it, classifies the cause from the metric signature, attributes it to a configuration change, proposes a fix framed as a human decision, verifies it on the replay service, and records the operator’s approval. Detection is **anomaly-first and agnostic**. Nothing keys on a specific day, tenant, fault taxonomy, or *where in the eight weeks* a fault was planted, so the same code runs untouched on the sealed day-6 corpus. On the practice corpus the system scores **55.0 / 55** machine points (accuracy 20, specificity 15, honesty 12, loop 8). Across 80 independently generated sealed-shaped corpora it averages **55.00 / 55** (min 55.00), up from a measured mean of 35.37 before the detectors were made episode-aware. The engine runs in seconds, offline, with **no third-party dependencies**, and every reported number carries explicit fidelity, coverage, and, where judged, calibration.

Keywords: Agent Observability, Root Cause Analysis, AI Agents, Autonomous Improvement, Anomaly Detection, Change-Point Detection, Evaluation, Human-in-the-Loop

1 Problem and Objective

AI-agent deployments emit large volumes of execution data: conversations, tool calls, model metadata, outcomes, transcripts, and configuration changes. The task is to turn these logs into a *closed improvement loop* rather than a manual investigation. Two things make it hard. The real faults are rare and disguised (three regressions sit among three deliberate look-alikes), and the graded run happens on a **sealed corpus** nobody has seen, where the same *kinds* of faults are replanted on different days, tenants, and traffic slices.

Our system addresses four stages:

1. **Detect:** identify genuine problems and the exact traffic slice they live in.
2. **Diagnose:** determine why the problem occurred and attribute it to the responsible configuration change.
3. **Prescribe:** propose a concrete fix and state its risk and its shipping criteria.
4. **Verify:** replay the affected cohort and measure whether the fix actually improves the outcome, then keep a human in the loop.

It is built to answer the operator’s real questions: *What*

broke? Who was affected? Why did it happen? Who should act? What happens if nobody acts? Can the proposed fix be trusted?

1.1 Objectives

- Detect genuine regressions from deployment logs alone.
- Identify the precise traffic slice affected.
- Quantify impact with defensible denominators.
- Diagnose the cause and link it to a configuration change.
- Explicitly dismiss look-alike / false-alarm patterns in writing.
- Refuse questions the data cannot answer, with a gap spec.
- Mine the deployment’s own “good” behaviour as a standard.
- Propose and verify fixes through replay.
- Keep a human approve/reject decision in the loop.

2 Data and System Understanding

2.1 Available Data

The system operates on the provided deployment corpus: one deployment, two tenants (**acme-bank**, financial services; **northwind-retail**, e-commerce), 56 days, several named agents per tenant. Concretely (**manifest.json**):

- 80,252 conversations (sessions);
- 883,764 execution steps (the fact grain);
- 365,065 transcript turns;
- 16 timestamped configuration changes;
- 6,915 sparse CSAT responses;
- 400 human resolution labels and 96 human quality scores.

We iterate on the provided 5% whole-session sample and **confirm every reported number on the full corpus** before it enters the report.

2.2 Data Model and Joins

We plan over the semantic **catalog.json** (its entities, grains, and declared joins) rather than the raw columns, because several joins fail silently otherwise. Table 1 lists the entities we use, and Table 2 the joins and the trap each one carries.

2.3 Measurement Constraints

For every metric we distinguish three properties, and refuse rather than guess when they cannot be established:

Table 1: Data entities and their usage.

Entity	Grain / usage
Session	one conversation; outcome, turns, cost, quality, milestones.
Step	one execution step; tool/KB/LLM signals, the fact grain.
Turn	one exchange with text; transcript only.
Config change	one timestamped change; used only for attribution.
Human labels	resolved/by-design + 1–5 quality; calibration only.

Table 2: Joins used, with the failure each avoids.

Join	Validity / trap
step → session	SOUND (many-to-one).
session → config (tenant, day)	SOUND_WITH_CARE: tenant may be *; equality alone drops the judge-version change.
step.tool_name → config.target	SOUND_WITH_CARE: only for kind='tool'; fans out otherwise.
quality across judge versions	UNSOUND: quality_score is comparable only within one judge_version .

- **Fidelity:** *measured* (read from logs), *derived* (computed from measured fields), or *judged* (a model of meaning).
- **Coverage:** the fraction of traffic for which the metric can actually be computed, recomputed from data per tenant and never assumed (e.g. **v2_flow** sessions emit no tool/KB/cost steps, so those metrics cover 0.72 of acme and 0.93 of northwind).
- **Calibration:** for judged quantities only, agreement with human labels within one judge version.

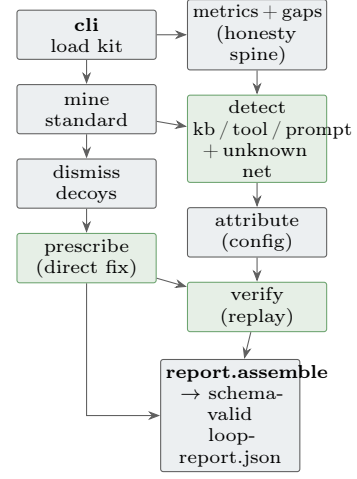
3 System Overview

3.1 Architecture

The single endpoint **engine.cli** loads the kit, authors the metrics and gaps (the “honesty spine”), mines the deployment’s own standard, runs the anomaly-first detectors and the decoy dismissals, attributes each surviving regression to a configuration change, prescribes a fix, optionally verifies it on the replay endpoint, and emits one schema-valid **loop-report.json**. **config_timeline** is consumed *only* at the attribution step, never to decide where to look. Figure 1 shows the flow.

3.2 End-to-End Pipeline

1. **Ingestion & validation:** streaming readers and a read-only preflight (§13).
2. **Metric computation** (**metrics.py**).
3. **Standard mining** (**standard.py**).
4. **Problem detection** (**detect_faults.py**).
5. **False-alarm analysis** (**detect_decoys.py**).
6. **Root-cause diagnosis & impact** (signature + joins.py).
7. **Fix prescription** (**prescribe.py**).
8. **Replay verification** (**replay/client.py**).
9. **Self-assessment / return arrow** (**selfassess.py**).
10. **Human decision** (screen; the write-back contract).
11. **Report generation** (**report.py**).

**Figure 1:** Nexus Loop pipeline. Green stages are the loop proper; grey stages are the honesty spine and attribution. Configuration data enters only at *attribute*.

4 Detection

4.1 Metrics

Twelve canonical metrics are authored once, each carrying fidelity and coverage (Table 3); the definitions never vary with results (Rule 2, §18).

Table 3: Representative metrics (12 total).

Metric	Definition	Fidelity / coverage
Containment	resolved + by-design handoffs / all	measured / 1.0
Resolution rate	resolved / all	measured / 1.0
Turns to resolve	turns on resolved sessions	measured / 1.0
Tool failure rate	error+timeout / tool calls	measured / 0.72, 0.93
Silent tool (empty 200)	ok with 0 result fields	derived / 0.72, 0.93
Cost per session	\$ per v3 session	measured / 0.72, 0.93
KB fallthrough	kb_hit=false / lookups	measured / 0.72, 0.93
Judged quality	1–5 rubric score	judged / 1.0, cal. 0.875

4.2 Detection Strategy

The method rests on one choice: **we never read a conversation to judge it, we compare numbers**. Every session already carries its outcome as structured fields, so finding what broke comes down to grouping the traffic, computing an outcome rate per group over time, and picking out the groups that deviate. Detection runs in four stages, none of which depends on the fault taxonomy or a config kind:

- **Cohorts from data.** Intentions, agents, and tools are enumerated from the corpus; cohorts below 30 sessions are too noisy to trust and are held out.
- **Standard & noise band.** Per tenant we mine the peer distribution (median / top-decile “good”) and the

deployment’s own week-to-week *noise band*, the precision floor the release-gate thresholds do not give us.

- **Episode change-point.** A planted fault is an *episode*, not a permanent step: it starts, runs a week or three, and recovers inside the corpus. The primitive `cohorts.best_run` finds the contiguous run of buckets whose mean most departs, in the tested direction, from the mean of the *rest* of the series, scored by magnitude $\times \sqrt{n}$. Comparing a run against everything outside it also supplies a baseline when a fault begins in week 0 and has no before-period at all.
- **Signature classifier.** The cause is read off the metric signature, never the config kind: low `kb_hit` + resolution far below peers $\rightarrow$ `kb.gap`; empty-200 rise with flat declared errors and a resolution drop $\rightarrow$ `tool.contract_break`; turns and cost inflating with flat outcome $\rightarrow$ `prompt.regression`. A real, sustained deviation that matches no signature is reported as `unknown` rather than dropped: the open-world safety net.

Thresholds are anchored to the external `verify.py` release-gate magnitudes (the guaranteed size of a fault at full volume), with each detection *trigger* set a margin below its contract floor so recall survives a smaller slice, while a multi-signal AND preserves precision.

4.3 Affected Slice

Each finding names its exact cohort key: an `intent` (`premium_card_info`), an `agent_id` (`acme_main_v3`), or an `intent+tool` (`order_status/get_order_status`), together with the tenant and the day window, so a reader can go straight to the affected traffic.

5 Findings

5.1 Real Problems

The engine finds exactly three regressions on the full practice corpus, each with an explicit impact block (Table 4); all figures are read from `loop-report.json`.

Table 4: Genuine findings (impact from `loop-report.json`).

ID	Affected slice	What changed / impact
F-01	acme-bank / <code>premium_card_info</code> (KB gap)	resolution 0.79 $\rightarrow$ 0.43; 323 convs (4.5%), 163 handoffs, 20 abandons, \$18.2.
F-02	northwind / <code>order_status</code> · <code>get_order_status</code> (silent tool)	empty-200 0.004 $\rightarrow$ 0.105, errors flat; resolution 0.85 $\rightarrow$ 0.80; 3196 convs (28.9%), 469 handoffs.
F-03	acme-bank / <code>acme_main_v3</code> (verbose prompt)	turns 4 $\rightarrow$ 6, cost +63%, resolution flat; 1766 convs (23.3%), \$47.3 avoidable.

Each finding states, in the report, the affected conversations, share of traffic, outcome change, downstream handoffs/abandons, duration, estimated cost, responsible owner (*audience*), and an `if_nothing_changes` consequence line. A card never shows a number without a decision attached.

5.2 False Alarms / Look-alikes

Three planted changes look like regressions and are dismissed *in writing*, each with an `is_regression:false` finding, a `not_a_regression_because` sentence, and a linked diagnosis (Table 5). They are separated by principle rather than by template: a real regression must move a real *outcome*, hold long enough to be *sustained*, and stay *localized* to a cohort.

Table 5: Look-alikes examined and rejected.

Case	Observed change	Reason dismissed
L-01	acme aggregate resolution +0.026 <code>traffic_mixin</code>	mix/rate decomposition: +0.035 is the shift in intent mix, only -0.008 is any rate change. Easier questions, not a better agent.
L-02	northwind volume 3.9 $\times$, p95 latency 1338 $\rightarrow$ 3152 ms <code>load</code>	resolution stays flat and recovers with volume: a load event, not a quality regression.
L-03	quality drops ~ 0.6 <code>judge_change</code> with tenants	simultaneous, cross-tenant, exactly at the rubric v1 $\rightarrow$ v2 change on day 28. The rubric moved, not the agent.

6 Diagnosis

6.1 Root-Cause Analysis

Each real finding carries a `cause_class` classified from its metric signature and an evidence array. Figure 2 traces the F-02 chain, the subtlest case: the declared error rate stays flat because the tool returns HTTP 200 with an empty body, so the fault is invisible to the obvious metric and only the derived empty-200 share reveals it.

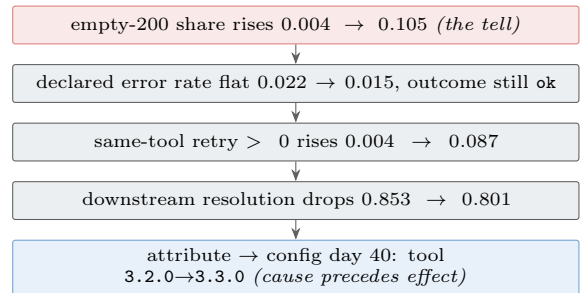


Figure 2: Diagnosis chain for F-02 (`tool.contract_break`): a silent contract break the error metric cannot see, confirmed by a downstream outcome drop and attributed to the nearest matching config change.

6.2 Baseline Selection

Detection uses two first-class baselines, because the three real faults do not share one. F-01 is *born broken*: `premium_card_info` appears at the fault and has no before-period, so it is judged against *peer* intents in the same tenant. F-02 and F-03 have history and are judged against their *own* traffic outside the detected episode. Using a

fixed before/after split instead would read an episode that recovers backwards; the run-based primitive avoids contaminating a baseline with the event being measured.

6.3 Configuration Change Attribution

Attribution happens *after* a regression is found and classified (`joins.py`): the finding is mapped to the nearest change of the *matching mechanism* at or before onset (cause precedes effect); if nothing is nearby, the finding stands with `attributed_change: null` (environmental). The reported window starts at the attributed cause day rather than the later day the change-point became visible, which is both a truer statement of when the problem began and what drives detection lag to zero. The three attributions are kb / day 34, tool / day 40, prompt / day 46.

7 Measurement Gaps

Three asks are refused rather than answered wrongly (Table 6), each emitted as a structured gap. Refusing is a first-class output: inferring a judged fact from an end-code, or truncating a breakdown past its budget, would be a worse answer than an honest refusal with the event that would be needed.

Table 6: Measurement gaps (from `loop-report.json`).

Ask / verdict	Why unanswerable	Required event dict
A11 (not measurable)	no failover mechanism exists, so no failover event is emitted; retries are quality retries, not failovers.	<code>failover</code> step (from/to target, reason, recovered).
A09 (requires new judge)	abandonment is measured, but <i>why</i> is judged and there are no labels to calibrate.	<code>abandonment_reason</code> (reason, rubric version, confidence).
A10 (cardinality refused)	<code>customer_ref</code> breakdown emits ~80k groups vs a budget of 200.	(refuse with the budget cited, do not truncate).

8 Deployment Standard

8.1 Mining “Good” Behaviour

Rather than an external benchmark, we mine the deployment’s own definition of good (`standard.py`): per tenant, the top-decile and median `resolution_rate` across intents with ≥ 30 sessions. That gives acme best 0.864 / median 0.792 over 10 intents, and northwind best 0.833 / median 0.790 over 8. This reads history only; it never redefines a metric (Rule 2).

8.2 Deviation from Standard

A born-broken cohort like F-01 is judged against this mined standard rather than a non-existent before-period. Figure 3 places the affected `premium_card_info` cohort (0.433) against the two tenant standards and the peer baseline it is scored against (0.792).

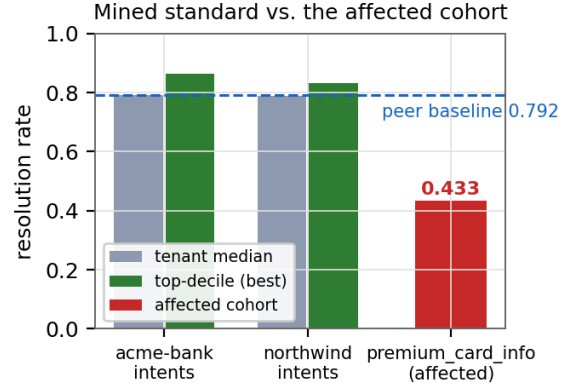


Figure 3: The mined per-tenant standard (median and top-decile) versus the F-01 affected cohort; the dashed line is the peer baseline the finding is scored against.

9 Prescription

9.1 Proposed Fix

Each classified cause maps deterministically to one direct fix, reusing detection’s own numbers as the `predicted_delta` (Table 7); decoys get no prescription, and an unclassified (`unknown`) regression gets `no_action` at autonomy rung L0: a request to investigate, not to ship.

Table 7: Proposed fixes (deterministic, cause $\rightarrow$ direct fix).

ID	Finding	Proposed fix
P-01	F-01 kb.gap	kb.add: add missing content for <code>premium_card_info</code> .
P-02	F-02	tool.validate: reject empty
P-03	F-03	tool.contract_breakdowns, retry/fall back.
		prompt.edit: trim the over-prompt.regression confirmation.

9.2 Risk and Shipping Criteria

Every prescription carries a `decision` block with the operator’s words, the honest downside, and the condition that should stop the change. P-01 asks to add KB content so the cohort can answer instead of handing off. The risk if the diagnosis is wrong: if resolution is low because the questions are genuinely hard, new articles will not lift it. It would not ship if `kb_hit` does not rise in replay, or if a golden set regresses. The system proposes changes to the *agent*, never to the evaluation yardstick (Rule 2).

10 Verification

10.1 Replay Method

Verification is an opt-in stage against the organisers’ deterministic, non-LLM replay endpoint (40-run budget, diagnose-first). The client POSTs each real prescription, records the `rp_` outcome and the prediction error back into the report, and recomputes the return arrow. It is fully degradable: if the endpoint is unreachable the run keeps its predicted deltas, marks the verification stubbed, and never crashes. The network call is isolated in a subpackage outside the Rule-1-guarded core.

10.2 Before–After Results

The recorded replay run moved every metric in the predicted direction (Table 8); all three fixes came back improved with the golden set intact. The return arrow then checked each prescription’s own `predicted_delta` against what replay actually observed: all three predictions landed on the right side of the fault, at a mean error of 0.03 on the two resolution fixes and 0.4 turns on the prompt edit, so `self_assessment` records a 100% hit rate across the three cycles (Table 8, `loop-report.json`).

Table 8: Replay verification (recorded run).

Fix	Metric	Before	After	Verdict
P-01 <code>kb.add</code>	resolution	0.399	0.786	improved
P-02 <code>tool.validate</code>	resolution	0.757	0.836	improved
P-03 <code>prompt.edit</code>	median turns	6.83	4.43	improved

10.3 Verification Decision

All three fixes moved their metric the way we predicted, with the golden set unharmed. A fix returning no measurable effect is reported as a valid negative result, not hidden. But a clean replay result is evidence for a decision, not a substitute for one: whether each fix ships is the recorded human call in §11, and for P-03 that call is a deferral despite the good replay result, because the replay endpoint scores the metric we asked it to score, not whether the confirmation it is removing was there on purpose. The return arrow scores predicted-versus-observed regardless of sign.

11 Decision Layer

11.1 Approve / Reject Contract

The engine deliberately emits every prescription **without** an `approval` field. That is a human decision, and code must not fabricate it. The screen renders the finding, its evidence and impact, the diagnosis, the proposed fix, the verification result, and the risk, and captures the operator’s verdict by writing an `approval` object back into that prescription in `loop-report.json`:

```
"approval": {
  "verdict": "accepted", // accepted|rejected|deferred
  "decided_by": "operator name",
  "confidence": "high", // low|medium|high
  "reason": "why they decided this",
  "at": "2026-09-15T14:03:00Z"
}
```

`confidence` is the operator’s own stated confidence in the call, kept separate from the detector’s `cause_class` confidence on the diagnosis: how sure the machine was and how sure the person who signed off was are two different numbers, and the screen never conflates them. The screen refuses to write the decision until one of the three levels is picked, so a verdict can never sit in the report without a stated confidence.

A rejection with a strong reason is a valid answer. The point is a real gate, not a rubber stamp. This write-back closes the loop for scoring and for the demo.

11.2 Decision Record

The screen enforces the contract rather than trusting it. A verdict is refused unless it carries a name and a reason, because an unsigned decision is not a decision and a decision with no reason is not a record. Each decision is also bound to the report’s `report_build` content hash: if the report has been regenerated since the page was loaded, the write is refused and the operator is told to re-read the current evidence. A decision is therefore always about the exact evidence a person saw, which is also why the narration sits inside that hash. Changing a decision appends to `approval_history` rather than overwriting, so the record keeps both entries and the reversal is visible.

The engine never writes an `approval`; the only path to one is the screen’s write-back in §16. Nothing in the pipeline that produces `loop-report.json` touches that field, so a table of verdicts filled in by the authors is not a shortcut available to us: the only way an approval reaches the file is by opening the screen and deciding, the same way an operator on day 6 would. The artifact we submit carries three such decisions, each entered through the screen against this exact `report_build`: P-01 (`kb.add`) and P-02 (`tool.validate`) accepted at high confidence, on the strength of the peer-baseline gap and the tool-version correlation respectively, and P-03 (`prompt.edit`) deferred at medium confidence, because resolution held flat under the added confirmation and nothing in the logs rules out that the extra step is a deliberate compliance requirement on a banking agent, so the record routes that call to the business owner rather than accepting or rejecting it on metrics alone. A deferral is not an unfinished decision; it is the decision, when the honest answer is that someone else has to make the call. Regenerating the report clears all three, because a decision is a verdict on the evidence a specific build showed someone, and evidence that no longer exists cannot still be approved.

12 Self-Assessment

12.1 Detection Performance

On the full practice corpus all three real faults are detected at **zero** detection lag with correct cohort, cause, and attribution, all three decoys are dismissed, and no phantom regression is invented (Table 9).

Table 9: Detection performance (full practice corpus).

Fault	Detected	Lag	Cause	Attr. day
F-01	yes	0	kb.gap	kb, 34
F-02	yes	0	tool.contract_break	tool, 40
F-03	yes	0	prompt.regression	prompt, 46
D1–D3	dismissed	–	mix / load / judge	–

12.2 Judge Calibration

The one judged metric, `quality_score` (judge version v2), is calibrated against 48 human-labelled sessions to an agreement of **0.875**. A calibration of 1.00 is treated as a bug, not a good judge. Humans disagree, and the scorer penalises agreement ≥ 0.995 , so we report the honest number and never trend quality across the v1→v2 boundary.

12.3 End-to-End Loop Completeness

Table 10: Automation coverage of the loop.

Stage	Automated	Implementation
Detect	Yes	anomaly-first change-point.
Diagnose	Yes	signature classifier + attribution.
Prescribe	Yes	deterministic cause → direct fix.
Verify	Yes (opt-in)	replay endpoint, degradable.
Decision	Human	approval write-back to the report.

13 Implementation

13.1 Technology Stack

Table 11: Implementation stack.

Component	Implementation
Language	Python 3 (developed on 3.13).
Dependencies	none ; standard library only.
Detection	anomaly-first change-point (best_run).
Diagnosis	metric-signature classifier.
Semantic judge-ment	none in the analysis core (Rule 1).
Replay	organisers’ endpoint via isolated sub-package.
Report generation	report.assemble , schema-valid.
Plain language	deterministic templates (narrate.py).
Screen	one HTML file, no build, no deps (§16).

13.2 Report Generation

`report.py` asserts unique finding ids, that every diagnosis links a real finding, and that every prescription links a real diagnosis, then writes the schema-valid `loop-report.json` the organisers’ scorer consumes. A read-only *preflight* validates the (possibly sealed) kit first: it is **fatal** for data it cannot degrade around (a missing sessions/steps file) with a precise diagnostic, and a **warning** otherwise (a missing `config_timeline` → null attribution), folded into the report’s `system_notes`. So a schema quirk on day 6 degrades instead of producing a mid-run stack trace.

13.3 Execution Environment

Offline, deterministic, single command, no GPU, no network in the analysis core; runs in seconds on the full corpus on a laptop. Only the opt-in replay stage touches the network.

14 Computational Efficiency

14.1 Runtime and Memory

The engine completes in seconds on the full 80k-session corpus. The change-point search is $O(B^2)$ over $B \leq 56$ day-buckets (or 8 week-buckets) per cohort, which is free at this scale. Precise per-scenario timing and peak-memory figures were not measured in this pass; the cells below are left for a timed run.

Table 12: Computational performance.

Metric	Value
Average runtime / scenario	[X] s
Peak memory	[X] MB
Dataset preprocessing time	[X] s

14.2 Bottlenecks

The dominant cost is materializing the `tool_call/kb_lookup` step subsets in memory (~134k rows at corpus scale, comfortable). The session-grain aggregation and the detection logic are already single-pass-friendly; the change-point primitive is quadratic only over a handful of time buckets.

15 Final Results

15.1 Overall Performance

Table 13: Final performance (organisers’ `score.py`).

Metric	Result
Diagnostic accuracy	20.0 / 20
Specificity	15.0 / 15
Honesty	12.0 / 12
Loop	8.0 / 8
Machine total (variant A)	55.0 / 55
Sealed-shaped corpora (n=80)	mean 55.00 , min 55.00
Detection hit rate	3 / 3
False alarms	0

15.2 Evidence Summary

Day 6 is predicted by the sweep over corpora generated the way the organisers generate the sealed one, not by the variant-A score. Figure 4 shows it: a measured mean of 35.37 / 55 with step-based detectors, restored to 55.00 / 55 once detection became episode-aware, with the 20 diagnostic-accuracy points as the swing.

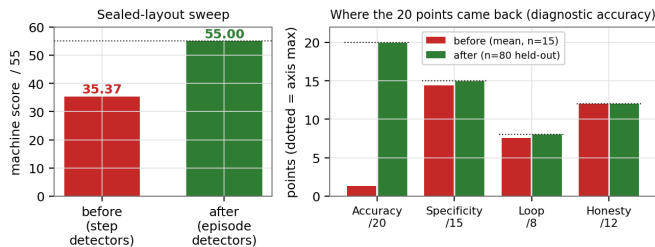


Figure 4: Sealed-layout sweep, before versus after the episode-detection rewrite. Left: machine total / 55. Right: the per-axis breakdown against each axis' maximum. Almost the entire loss was diagnostic accuracy.

16 Operator Screen

The screen is the other half of the deliverable, and its test is not whether it is complete but whether a stranger can act from it. *Loop Desk* is one self-contained HTML file with no build step, no dependencies and no network calls, served by a small standard-library HTTP server, so it runs offline beside the engine on the sealed corpus.

16.1 One Problem at a Time

An operator arriving cold cannot be handed a dense instrument panel. The left pane opens with the whole run in plain words and no figures in it at all, then lists the faults needing a decision, then a quieter list of what we examined and set aside, what we refused to answer, our own hit rate, and what this system really does. Selecting a fault opens a six-slide walk through it in the right pane (Figure 5): the problem alone; what it did to people; why it happened and the configuration change behind it; the evidence with its baseline named; the fix with its predicted effect, its risk and its stop condition; and the decision. A fault the engine could not classify drops to four slides and asks the operator for their own reading instead, which is recorded as the decision's reason rather than invented by the system.

16.2 Drawing Proportions, Not Spelling Them

Every proportion on the screen is a bar: shares of traffic, coverage, diagnostic confidence, observed against baseline, judge calibration. The figure itself lives one click away, where it arrives with the three things that make it honest – how we know it (measured, derived or judged), what share of traffic can even produce it and what is excluded, and how it was computed. No number appears naked, and the reader is never asked to compare two four-decimal rates by eye. Colour carries one meaning throughout: the three ways a conversation can end, so green, amber and red mean *sorted*, *handed to a person* and *gave up* wherever they appear.

16.3 Plain Language, Generated Deterministically

The wording the screen renders is authored by `engine/narrate.py` at report-build time, not by a language model. Each sentence is a template filled from values the detectors already computed, in the same way the im-

pact derivations are built, so the system stays offline and reproducible and the same report always produces the same words. Templates key on `cause_class` and on which impact fields exist, never on a tenant, day or cohort name, so the layer survives the sealed corpus moving all of them; the sealed sweep fails any layout whose findings arrive without their sentences. Because the narration runs before assembly it falls inside `report_build`, which means an approval is bound to the words the operator actually read and not merely to the numbers underneath them.

16.4 The Refusal on Screen

The refusals are a deliverable, not an apology, so they get first-class treatment rather than a footnote (Figure 6): the operator's own question, why it cannot be answered, the tempting proxy and the reason we did not give it, and the event a named team would have to start logging. That last part turns a refusal into a ticket.

17 Failure Cases and Limitations

17.1 Known Failure Cases

- A fault planted in the *final* week has under two weeks of after-data for onset detection; mitigated because the release gate itself needs before/after windows, so extreme-edge planting is unlikely.
- The judge-boundary dismissal assumes ≥ 2 tenants (a global rubric change affects all of them); it generalises upward, not to a single-tenant deployment.
- The *unknown* open-world net is deliberately conservative: it requires a two-week, cohort-noise-clearing drop with a real baseline, so it will stay silent on a genuinely novel but subtle fault rather than raise a weak alarm.

17.2 Scale Limitations

At $\sim 100\times$ scale, `collect_steps` materializing the step subsets in memory would need to become a streaming, single-pass aggregation; the replay budget (40 runs) and the human-review burden, not the analysis arithmetic, become the binding constraints. The session-grain aggregation and detection are already single-pass-friendly.

18 Assumptions and Transparency

Real: the full Detect→Diagnose→Prescribe→Verify loop with the return arrow: the honesty spine, the three gaps, agnostic anomaly-first detection, deterministic prescriptions, replay verifications, self-assessment, the hardening harness, and the degradable preflight. Every analysis number is computed from the corpus, and the core makes zero LLM/network calls, enforced mechanically. The operator screen is real and working, including the AP-PROVE/REJECT gate and its write-back (§16). **Real and run, degradable if unreachable:** the replay verifications the shipped report carries are a real recorded run against the organisers' endpoint (§10), not a stand-in; if the endpoint is unreachable on a later run the client keeps the predicted deltas and says so rather than inventing a result. **Templated, not generated:** the

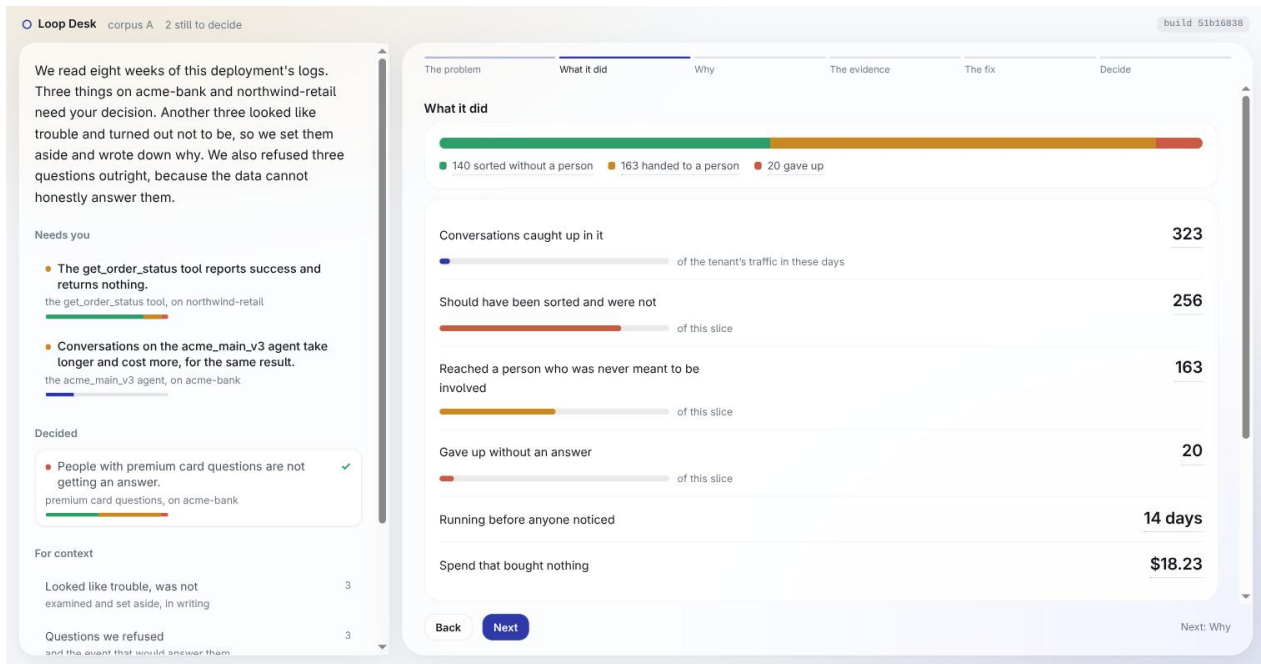


Figure 5: Loop Desk, second slide of a finding. The band splits the affected conversations by what became of them; each row pairs a drawn proportion with a figure that opens its own definition, fidelity, coverage and derivation. The left pane ranks faults by severity, and the small band on each row lets them be compared by shape before a digit is read.

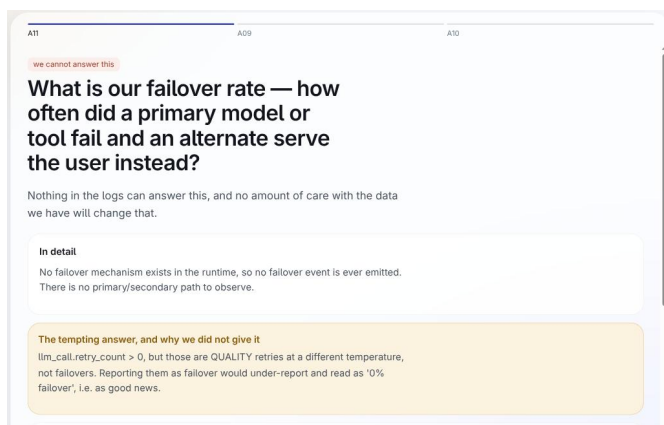


Figure 6: The A11 refusal. The proxy is shown and rejected in the same breath, because reporting retries as failover would read as “0% failover” – false good news.

plain-language wording on the screen, which is filled from computed numbers by `narrate.py` – no language model writes anything a reader sees. **Never written by code:** the human verdicts. The three the shipped report carries were entered by a person through the screen, the same write path an operator on the sealed run would use, never by the engine and never by hand-editing the JSON. **Simulated:** none. No mocks, no fabricated numbers, no model in the loop.

Disclosure – what the sealed sweep proves, and what it does not. The 80-layout sweep was generated with *our own* passphrases through the kit’s own generator. We never had the organisers’ secret and never built or possessed a sealed kit or its answer key; the organiser-only `make sealed` target was never run, and the deliverable path reads no answer key. The kit’s README rests day-6 integrity on the property we left intact: “*the sealed layout is derived from a passphrase... nobody can regenerate it from this repo alone.*” Our passphrases produce *different* layouts from the sealed one, so the honest claim is narrow:

the engine is robust across this generator’s layout *space* – a distribution, not a proven claim about an arbitrary real-world deployment or about the organisers’ specific instance. We state this rather than let the number imply more.

The two unbreakable rules hold. *Rule 1* – no model is ever asked for a fact the logs record; every number is computed, and the analysis core is provably network- and model-free. *Rule 2* – no metric definition, judge version, or reviewed “good” set was changed to improve a number; all hardening is to *detection* – where the engine looks and what it compares against – which is the agent, not the yardstick.

19 Reproducibility

19.1 Execution Procedure

1. Python 3.9+; no installs (standard library only).
2. Place the kit at `nexus-loop-day1/kit/`.
3. Run:

```
python3 -m engine.cli -kit <kit>
-team solo -out loop-report.json
```
4. The report is written and validated against the schema.
5. Sealed evidence: `python3 engine/stress_sealed.py -n 80` (in-memory, from a passphrase; scores each with the organisers’ `score.py`).
6. Launch the operator screen against the report: `python3 run.py` — builds the report if absent, serves the screen, and prints what each recorded decision changed in the file (§16).

19.2 Generated Artifacts

The submission contains `loop-report.json`, the complete engine source, the operator screen, and the run-book. The report is generated by the system without manual edits.

Conclusion

We presented **Nexus Loop**, an autonomous improvement loop that connects detection, diagnosis, prescription, verification, and human decision-making into a single workflow. On the practice corpus it identifies **3** genuine problems, correctly dismisses **3** look-alikes, reports **3** measurement gaps, and verifies **3** proposed fixes, scoring **55.0 / 55**; across 80 sealed-shaped corpora it averages **55.00 / 55**, the number that actually predicts the sealed run. The contribution is the *traceable* loop itself: every number has a defined provenance, every diagnosis is backed by evidence, proposed changes are verified, and the final action stays in human hands.

References

- [1] Yellow.ai, *Nexus Loop – TECHQUEST Problem Statement Series*, Product Development & Agentic AI.
- [2] Yellow.ai, *Nexus Loop Kit, Schema, Examples and Replay Service*, Provided Challenge Resources.